

AWS Architect | Interview Preparation

Position: AWS Architect
Location: Hyderabad
Experience: 7-12 years
Key Skills: AWS Network, Infrastructure as Code, Terraform, GitHub, GitHub Actions
Prepared for: Pushparaj Naik | 22+ Years Experience

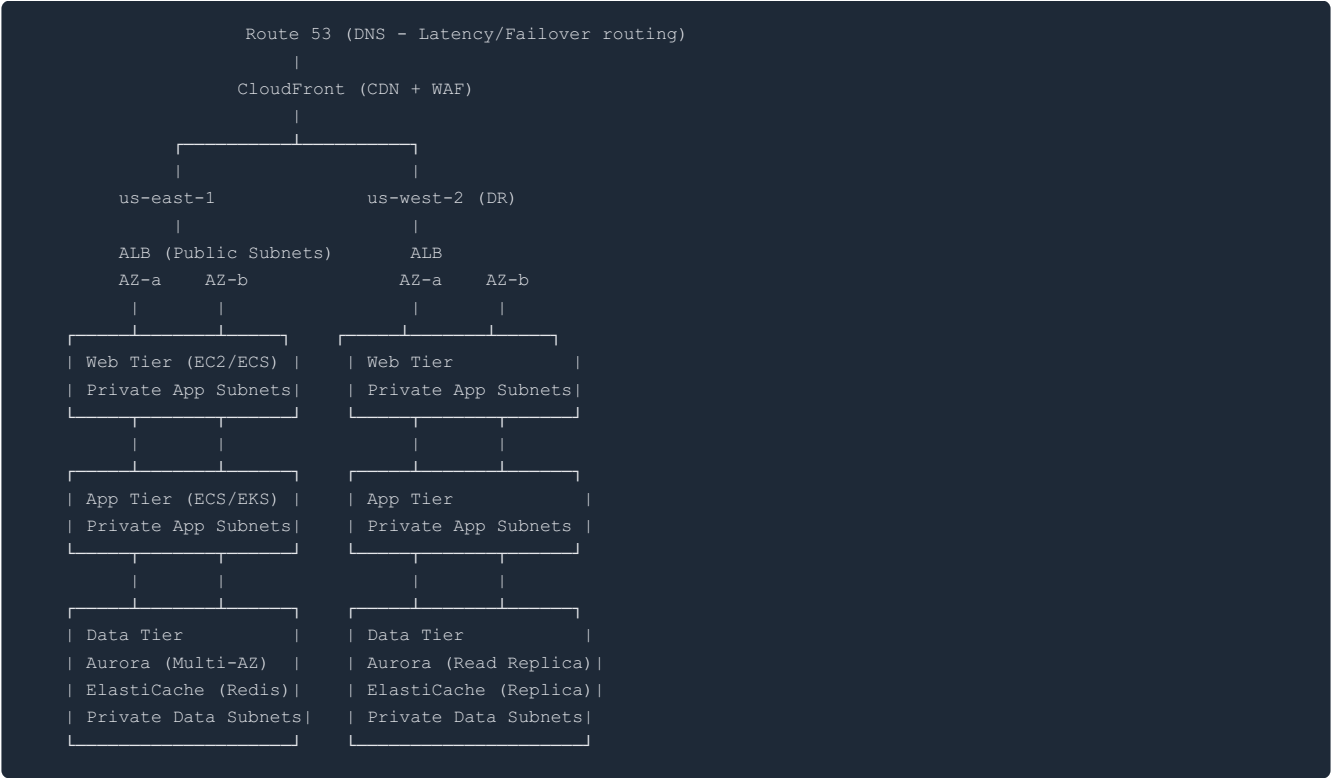
Table of Contents

- 1. [Cloud Architecture Leadership](#)
- 2. [Infrastructure as Code \(Terraform\)](#)
- 3. [CI/CD Pipeline Integration \(GitHub Actions\)](#)
- 4. [GitOps & Version Control](#)
- 5. [AWS Networking Expertise](#)
- 6. [Security & Compliance](#)
- 7. [Cost Optimization](#)
- 8. [Multi-Environment Management](#)
- 9. [Cross-Functional Collaboration](#)
- 10. [Mentorship & Innovation](#)
- 11. [Scenario-Based Questions](#)
- 12. [Quick Reference Cheat Sheet](#)

1. Cloud Architecture Leadership

Q1: Design a secure, scalable, and highly available AWS architecture for a 3-tier web application.

Answer:



Key Design Decisions:

Decision	Choice	Rationale
High Availability	Multi-AZ in 2 regions	Survives AZ failure AND region failure
Scalability	Auto Scaling Groups + ALB	Horizontal scaling based on CPU/request count
Security	Private subnets + NAT	No direct internet access to app/data tiers
Data	Aurora Multi-AZ + Global Database	Automatic failover, cross-region replication
Caching	ElastiCache Redis (cluster mode)	Sub-millisecond reads, session management
Edge	CloudFront + WAF	DDoS protection, SSL termination, caching
DNS	Route 53 failover routing	Automatic failover to DR region

Q2: Explain VPC architecture in detail. How would you design a production VPC?

Answer:

VPC Design:

```
VPC: 10.0.0.0/16 (65,536 IPs)
├── Public Subnets (for internet-facing resources)
│   ├── 10.0.0.0/24 (AZ-a) – ALB, NAT Gateway, Bastion
│   └── 10.0.1.0/24 (AZ-b) – ALB, NAT Gateway
├── Private App Subnets (for compute)
│   ├── 10.0.10.0/24 (AZ-a) – EC2, ECS, EKS worker nodes
│   └── 10.0.11.0/24 (AZ-b) – EC2, ECS, EKS worker nodes
├── Private Data Subnets (for databases)
│   ├── 10.0.20.0/24 (AZ-a) – RDS, ElastiCache, OpenSearch
│   └── 10.0.21.0/24 (AZ-b) – RDS, ElastiCache, OpenSearch
├── Internet Gateway (attached to VPC)
├── NAT Gateways (1 per AZ in public subnet – for HA)
├── Route Tables
│   ├── Public RT: 0.0.0.0/0 → IGW
│   ├── Private RT: 0.0.0.0/0 → NAT Gateway
│   └── Data RT: No internet route (isolated)
└── VPC Endpoints
    ├── S3 (Gateway endpoint – free)
    ├── DynamoDB (Gateway endpoint – free)
    └── ECR, STS, Logs, SSM, KMS (Interface endpoints)
```

Key Concepts:

Component	Purpose	Key Details
Subnet	Partition within an AZ	Each subnet maps to exactly 1 AZ. Cannot span AZs.
Route Table	Controls traffic routing	Each subnet associated with 1 route table. Most specific route wins.
Internet Gateway	Connects VPC to internet	Horizontally scaled, HA. Attached at VPC level, used via route table.
NAT Gateway	Allows private subnets to reach internet	Needed for patching, pulling images. One per AZ for HA.
Security Group	Instance-level firewall (stateful)	Allow rules only. Return traffic automatically allowed.
NACL	Subnet-level firewall (stateless)	Allow AND deny rules. Must explicitly allow return traffic.
VPC Endpoint	Private access to AWS services	Avoids NAT Gateway charges. Stays within AWS network.

Q3: What is the difference between Security Groups and NACLs?

Answer:

Feature	Security Group	NACL
Level	Instance/ENI level	Subnet level
Stateful/Stateless	Stateful (return traffic auto-allowed)	Stateless (must define inbound AND outbound rules)
Rules	Allow only (implicit deny all)	Allow AND Deny
Evaluation	All rules evaluated together	Rules evaluated in order (lowest number first)
Default	Deny all inbound, allow all outbound	Allow all inbound and outbound
Association	Multiple SGs per instance	1 NACL per subnet (shared by all instances)
Use case	Primary firewall for instances	Additional subnet-level defense layer

Example — Web Server:

```
Security Group (sg-web):
  Inbound:  HTTPS (443) from ALB security group
  Outbound: All traffic (implicit allow)

NACL (public subnet):
  Inbound Rule 100: Allow HTTPS (443) from 0.0.0.0/0
  Inbound Rule 200: Allow ephemeral ports (1024-65535) from 0.0.0.0/0
  Inbound Rule *:   Deny all
  Outbound Rule 100: Allow HTTPS (443) to 0.0.0.0/0
  Outbound Rule 200: Allow ephemeral ports (1024-65535) to 0.0.0.0/0
  Outbound Rule *:   Deny all
```

Q4: Explain subnets, route tables, and how traffic flows within a VPC.

Answer:

How Traffic Flows:

```
Internet User → Route 53 → CloudFront → ALB (Public Subnet)
                                     |
                                     Route Table says: "10.0.10.0/24 → local"
                                     |
                                     EC2 in Private App Subnet
                                     |
                                     Route Table says: "10.0.20.0/24 → local"
                                     |
                                     RDS in Private Data Subnet

EC2 needs internet (for patching):
  EC2 → Route Table: "0.0.0.0/0 → NAT Gateway" → NAT GW → IGW → Internet
```

Route Table Rules:

Destination	Target	Meaning
10.0.0.0/16	local	Traffic within VPC stays within VPC (auto-added)
0.0.0.0/0	igw-xxx	All other traffic goes to Internet Gateway (public RT)
0.0.0.0/0	nat-xxx	All other traffic goes to NAT Gateway (private RT)
p1-xxx (S3 prefix list)	vpce-xxx	S3 traffic goes through VPC endpoint
10.1.0.0/16	pcx-xxx	Traffic to peered VPC goes through peering connection
10.2.0.0/16	tgw-xxx	Traffic to another VPC goes through Transit Gateway

Subnet Key Facts: - A subnet exists in exactly **one Availability Zone** — it cannot span AZs - A subnet is "public" if its route table has a route to an Internet Gateway - A subnet is "private" if it does NOT have a route to an IGW (may have NAT) - AWS reserves **5 IPs** per subnet (first 4 + last 1): network, VPC router, DNS, future use, broadcast - A /24 subnet = 256 IPs - 5 reserved = **251 usable IPs**

Q5: What are Availability Zones and how do you design for high availability?

Answer:

Availability Zone (AZ): A physically isolated data center (or group of data centers) within an AWS Region. Each AZ has independent power, cooling, and networking. AZs within a region are connected via low-latency links (< 2ms).

Design for HA:

```
Region: us-east-1
├── AZ: us-east-1a
│   ├── ALB node (auto-distributed)
│   ├── EC2 instances (ASG min=2)
│   ├── Aurora Writer
│   ├── ElastiCache Primary
│   └── NAT Gateway
└── AZ: us-east-1b
    ├── ALB node (auto-distributed)
    ├── EC2 instances (ASG min=2)
    ├── Aurora Reader (auto-failover to writer)
    ├── ElastiCache Replica
    └── NAT Gateway
```

HA Best Practices:

Resource	HA Strategy	Failover Time
EC2	Auto Scaling Group across 2+ AZs	~2-3 minutes (launch new)
ALB	Automatically multi-AZ	Instant (built-in)
Aurora	Multi-AZ with auto-failover	~30 seconds
ElastiCache	Multi-AZ with automatic failover	~30-60 seconds
NAT Gateway	One per AZ (not cross-AZ)	Zonal (isolated per AZ)
S3	Automatically replicated across 3+ AZs	N/A (built-in 99.99999999%)
DynamoDB	Automatically replicated across 3 AZs	N/A (built-in)

2. Infrastructure as Code (Terraform)

Q6: Explain Terraform architecture and workflow. What is state and why is it important?

Answer:

Terraform Workflow:

```
terraform init    → Download providers, initialize backend
|
terraform plan    → Compare desired state (code) vs current state (state file)
|                  → Show what will be created/changed/destroyed
|
terraform apply   → Execute the plan, create/modify/destroy resources
|                  → Update state file
|
terraform destroy → Remove all resources managed by this configuration
```

State File (terraform.tfstate):

The state file is a JSON file that maps your Terraform resources to real-world infrastructure.

Why it's critical: 1. **Tracking:** Maps `aws_instance.web` in code to `i-0abc123` in AWS 2. **Dependencies:** Knows the order to create/destroy resources 3. **Performance:** Caches resource attributes to avoid querying AWS APIs for every plan 4. **Collaboration:** When stored remotely (S3), allows team members to share state

Remote State Best Practices:

```
# backend.tf
terraform {
  backend "s3" {
    bucket     = "company-terraform-state"
    key        = "production/networking/terraform.tfstate"
    region     = "us-east-1"
    encrypt    = true                    # SSE-S3 encryption
    kms_key_id = "alias/terraform-state" # KMS encryption
    dynamodb_table = "terraform-lock"   # State locking
  }
}
```

Best Practice	Why
S3 backend (not local)	Team collaboration, durability
DynamoDB locking	Prevents concurrent state modifications
Encryption (KMS)	State contains sensitive data (passwords, keys)
Versioning on S3	Rollback state if corrupted
Separate state per environment	Blast radius — dev change can't corrupt prod state

Q7: How do you structure Terraform for a large organization with multiple environments?

Answer:

Directory Structure:

```
terraform/
├── modules/                                # Reusable modules
│   ├── networking/
│   │   ├── main.tf                        # VPC, subnets, route tables, NAT
│   │   ├── variables.tf                  # Inputs: CIDR, AZs, tags
│   │   ├── outputs.tf                   # Outputs: VPC ID, subnet IDs
│   │   └── versions.tf                  # Required providers
│   ├── compute/
│   │   ├── main.tf                      # EC2, ASG, ALB, target groups
│   │   ├── variables.tf
│   │   └── outputs.tf
│   ├── database/
│   │   ├── main.tf                      # RDS, ElastiCache
│   │   ├── variables.tf
│   │   └── outputs.tf
│   └── security/
│       ├── main.tf                      # IAM roles, policies, KMS
│       ├── variables.tf
│       └── outputs.tf
├── environments/
│   ├── dev/
│   │   ├── main.tf                      # module "networking" { source = "../../modules/networking" }
│   │   ├── variables.tf
│   │   ├── terraform.tfvars            # Dev-specific values (small instances)
│   │   ├── backend.tf                  # S3 backend: key = "dev/terraform.tfstate"
│   │   └── providers.tf
│   ├── staging/
│   │   ├── main.tf                      # Same modules, different tfvars
│   │   ├── terraform.tfvars            # Staging-specific values
│   │   └── backend.tf                  # key = "staging/terraform.tfstate"
│   └── production/
│       ├── main.tf                      # Prod-specific values (large instances, multi-AZ)
│       ├── terraform.tfvars
│       └── backend.tf                  # key = "production/terraform.tfstate"
└── bootstrap/
    ├── main.tf                          # One-time setup
    └── outputs.tf                        # S3 bucket + DynamoDB table for state
```

Key Principles:

1. **DRY (Don't Repeat Yourself):** Modules define resources once; environments pass different variables.
2. **Separate state per environment:** Dev, staging, prod have independent state files.
3. **Module versioning:** Use tagged Git releases for modules:

```
hcl module "networking" { source = "git::https://github.com/company/tf-modules.git//networking?ref=v2.1.0" }
```

4. **Variable validation:**

```
hcl variable "environment" { type = string validation { condition = contains(["dev", "staging", "production"], var.environment) error_message = "Environment must be dev, staging, or production." } }
```

Q8: What are Terraform modules? How do you write a reusable VPC module?

Answer:

A module is a container for multiple resources that are used together. It takes inputs (variables), creates resources, and produces outputs.

Example: Reusable VPC Module

```
# modules/networking/variables.tf
variable "project_name"    { type = string }
variable "environment"     { type = string }
variable "vpc_cidr"        { type = string, default = "10.0.0.0/16" }
variable "azs"             { type = list(string), default = ["us-east-1a", "us-east-1b"] }
variable "public_subnets" { type = list(string), default = ["10.0.0.0/24", "10.0.1.0/24"] }
variable "private_subnets" { type = list(string), default = ["10.0.10.0/24", "10.0.11.0/24"] }
variable "database_subnets" { type = list(string), default = ["10.0.20.0/24", "10.0.21.0/24"] }
variable "enable_nat"      { type = bool, default = true }

# modules/networking/main.tf
resource "aws_vpc" "main" {
  cidr_block           = var.vpc_cidr
  enable_dns_support   = true
  enable_dns_hostnames = true
  tags = {
    Name           = "${var.project_name}-${var.environment}-vpc"
    Environment    = var.environment
    ManagedBy      = "terraform"
  }
}

resource "aws_subnet" "public" {
  count                = length(var.public_subnets)
  vpc_id              = aws_vpc.main.id
  cidr_block          = var.public_subnets[count.index]
  availability_zone    = var.azs[count.index]
  tags = { Name = "${var.project_name}-${var.environment}-public-${var.azs[count.index]}" }
}

resource "aws_subnet" "private" {
  count                = length(var.private_subnets)
  vpc_id              = aws_vpc.main.id
  cidr_block          = var.private_subnets[count.index]
  availability_zone    = var.azs[count.index]
  tags = { Name = "${var.project_name}-${var.environment}-private-${var.azs[count.index]}" }
}

resource "aws_internet_gateway" "main" {
  vpc_id = aws_vpc.main.id
}

resource "aws_nat_gateway" "main" {
  count                = var.enable_nat ? length(var.azs) : 0
  allocation_id        = aws_eip.nat[count.index].id
  subnet_id            = aws_subnet.public[count.index].id
}

# modules/networking/outputs.tf
output "vpc_id" { value = aws_vpc.main.id }
output "public_subnet_ids" { value = aws_subnet.public[*].id }
output "private_subnet_ids" { value = aws_subnet.private[*].id }
```

Using the module:

```
# environments/production/main.tf
module "networking" {
  source      = "../../modules/networking"
  project_name = "myapp"
  environment  = "production"
  vpc_cidr     = "10.0.0.0/16"
  azs         = ["us-east-1a", "us-east-1b"]
  public_subnets = ["10.0.0.0/24", "10.0.1.0/24"]
  private_subnets = ["10.0.10.0/24", "10.0.11.0/24"]
  enable_nat     = true
}
```

Q9: Explain Terraform state locking, state drift, and how you handle them.

Answer:

State Locking: - When someone runs `terraform apply`, Terraform acquires a lock on the state file via DynamoDB. - Prevents two people from modifying the same infrastructure simultaneously. - Lock is released when the operation completes. - If a lock is stuck (process crashed), use `terraform force-unlock <LOCK_ID>`.

State Drift: State drift occurs when the actual infrastructure differs from the state file. Causes: 1. Manual changes in AWS console (someone added a security group rule manually) 2. Another tool modified the resource (CloudFormation, CLI) 3. AWS auto-remediation (Security Hub auto-fixing)

Detecting Drift:

```
# Run plan regularly (nightly via CI/CD)
terraform plan -detailed-exitcode
# Exit code 0: No changes
# Exit code 1: Error
# Exit code 2: Changes detected (DRIFT!)
```

Handling Drift:

```
# Option 1: Refresh state to match reality (accept the manual change)
terraform apply -refresh-only

# Option 2: Overwrite manual change (enforce IaC as source of truth)
terraform apply # Will revert manual change to match code

# Option 3: Import the manually created resource into state
terraform import aws_security_group.manual sg-0abc123
```

Best Practice: Schedule nightly `terraform plan` via GitHub Actions. Alert if drift detected. Enforce policy: all changes via Terraform PRs only; console access is read-only for production.

Q10: What is `terraform import` and when do you use it?

Answer:

`terraform import` brings existing AWS resources under Terraform management without recreating them.

When to use: - Migrating manually-created infrastructure to IaC - Someone created a resource in the console that should be managed by Terraform - Adopting Terraform in an existing AWS environment

Process:

```

# Step 1: Write the resource block in Terraform (empty or estimated)
# main.tf
resource "aws_vpc" "existing" {
  # Will be filled after import
}

# Step 2: Import the resource
terraform import aws_vpc.existing vpc-0abc123def456

# Step 3: Run terraform plan to see what attributes differ
terraform plan
# Output shows what needs to be added to the resource block

# Step 4: Update the resource block to match the actual configuration
resource "aws_vpc" "existing" {
  cidr_block      = "10.0.0.0/16"
  enable_dns_support = true
  enable_dns_hostnames = true
  tags = {
    Name = "production-vpc"
  }
}

# Step 5: Verify - plan should show "No changes"
terraform plan

```

Bulk Import (Terraform 1.5+):

```

import {
  to = aws_vpc.existing
  id = "vpc-0abc123def456"
}

import {
  to = aws_subnet.public_a
  id = "subnet-0abc123"
}

```

Then run `terraform plan -generate-config-out=generated.tf` to auto-generate the HCL.

Q11: Explain `count` vs `for_each` in Terraform. When do you use each?

Answer:

Feature	<code>count</code>	<code>for_each</code>
Input	Integer	Map or set of strings
Reference	<code>resource.name[0]</code> , <code>resource.name[1]</code>	<code>resource.name["key"]</code>
Best for	Creating N identical resources	Creating resources from a map/set
Reorder safe?	NO — removing item 0 shifts all indices	YES — removing a key only affects that resource

Problem with `count` :

```

# If you remove "us-east-1a" from the list, Terraform will DESTROY
# and RECREATE subnet at index [1] because indices shift!
variable "azs" { default = ["us-east-1a", "us-east-1b", "us-east-1c"] }

resource "aws_subnet" "private" {
  count          = length(var.azs)
  availability_zone = var.azs[count.index] # index-based = fragile
}

```

Solution with `for_each` :


```
# Removing "us-east-1a" only destroys THAT subnet. Others unaffected.
variable "subnets" {
  default = {
    "us-east-1a" = "10.0.10.0/24"
    "us-east-1b" = "10.0.11.0/24"
    "us-east-1c" = "10.0.12.0/24"
  }
}

resource "aws_subnet" "private" {
  for_each      = var.subnets
  vpc_id        = aws_vpc.main.id
  cidr_block    = each.value
  availability_zone = each.key
}

# Reference: aws_subnet.private["us-east-1a"].id
```

Rule of thumb: Use `for_each` for everything. Only use `count` for simple on/off toggles:

```
resource "aws_nat_gateway" "main" {
  count = var.enable_nat ? 1 : 0 # This is fine
}
```

Q12: What are Terraform workspaces? Do you recommend them?

Answer:

Workspaces allow you to maintain multiple state files within the same configuration.

```
terraform workspace new dev
terraform workspace new staging
terraform workspace new production
terraform workspace select production
terraform apply
```

Each workspace gets its own state file: `terraform.tfstate.d/dev/terraform.tfstate`

My Recommendation: I do NOT recommend workspaces for environment management.

Approach	Workspaces	Separate Directories
State isolation	Same backend, different state files	Completely separate backends
Config differences	Conditional logic: <code>count = terraform.workspace == "prod" ? 3 : 1</code>	Different <code>terraform.tfvars</code> per env
Risk	Easy to accidentally apply to wrong workspace	Directory name makes env obvious
Visibility	Hidden — <code>terraform workspace show</code> to check	Visible in path: <code>cd environments/production</code>
IAM	Same credentials for all envs	Different IAM roles per env (least privilege)

When workspaces ARE useful: For ephemeral environments (feature branch environments, PR previews).

3. CI/CD Pipeline Integration (GitHub Actions)

Q13: Design a Terraform CI/CD pipeline using GitHub Actions.

Answer:

```

# .github/workflows/terraform.yml
name: Terraform CI/CD

on:
  pull_request:
    paths: ['terraform/**']
  push:
    branches: [main]
    paths: ['terraform/**']

permissions:
  id-token: write      # OIDC authentication
  contents: read
  pull-requests: write # Comment plan output on PR

env:
  TF_VERSION: "1.7.0"
  WORKING_DIR: "terraform/environments/production"

jobs:
  # — Job 1: Security Scan —
  security:
    name: Security Scan
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Run Checkov
        uses: bridgecrewio/checkov-action@v12
        with:
          directory: terraform/
          framework: terraform
          soft_fail: false # Fail pipeline on CRITICAL

      - name: Run tfsec
        uses: aquasecurity/tfsec-action@v1.0.3
        with:
          working_directory: terraform/

  # — Job 2: Terraform Plan (on PR) —
  plan:
    name: Terraform Plan
    needs: security
    runs-on: ubuntu-latest
    if: github.event_name == 'pull_request'
    steps:
      - uses: actions/checkout@v4

      - name: Configure AWS Credentials (OIDC)
        uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: ${ secrets.AWS_ROLE_ARN }
          aws-region: us-east-1

      - name: Setup Terraform
        uses: hashicorp/setup-terraform@v3
        with:
          terraform_version: ${ env.TF_VERSION }

      - name: Terraform Init
        run: terraform init
        working-directory: ${ env.WORKING_DIR }

      - name: Terraform Validate
        run: terraform validate
        working-directory: ${ env.WORKING_DIR }

      - name: Terraform Plan
        id: plan
        run: terraform plan -no-color -out=tfplan
        working-directory: ${ env.WORKING_DIR }

      - name: Comment Plan on PR

```

```

uses: actions/github-script@v7
with:
  script: |
    const output = `#### Terraform Plan
    \`\`\`
    ${ steps.plan.outputs.stdout }
    \`\`\`
    *Pushed by: @${ github.actor }*`;
    github.rest.issues.createComment({
      issue_number: context.issue.number,
      owner: context.repo.owner,
      repo: context.repo.repo,
      body: output
    })

# — Job 3: Terraform Apply (on merge to main) —
apply:
  name: Terraform Apply
  needs: security
  runs-on: ubuntu-latest
  if: github.ref == 'refs/heads/main' && github.event_name == 'push'
  environment: production # Requires manual approval
  steps:
    - uses: actions/checkout@v4

    - name: Configure AWS Credentials
      uses: aws-actions/configure-aws-credentials@v4
      with:
        role-to-assume: ${ secrets.AWS_ROLE_ARN }
        aws-region: us-east-1

    - name: Setup Terraform
      uses: hashicorp/setup-terraform@v3

    - name: Terraform Init
      run: terraform init
      working-directory: ${ env.WORKING_DIR }

    - name: Terraform Apply
      run: terraform apply -auto-approve
      working-directory: ${ env.WORKING_DIR }

```

Key Design Decisions:

Decision	Why
OIDC auth (not access keys)	No long-lived secrets. Token valid for 1 hour only.
Checkov/tfsec before plan	Catch security issues before infrastructure changes
Plan on PR, Apply on merge	Review before apply. No direct applies.
Plan output as PR comment	Reviewers see exactly what will change
Environment protection	<code>production</code> environment requires manual approval
<code>-out=tfplan</code>	Ensures the exact plan reviewed is what gets applied

Q14: How do you authenticate GitHub Actions with AWS without storing access keys?

Answer:

OIDC Federation (recommended approach):

```

GitHub Actions Runner
|
| (1) Requests OIDC token from GitHub
v
GitHub OIDC Provider (token.actions.githubusercontent.com)
|
| (2) Sends JWT token
v
AWS IAM Identity Provider (OIDC)
|
| (3) Validates JWT, issues temporary credentials
v
AWS STS (AssumeRoleWithWebIdentity)
|
| (4) Returns temporary access key, secret, session token (1 hour)
v
Terraform uses temporary credentials

```

AWS Setup (Terraform):

```

# Create OIDC provider for GitHub
resource "aws_iam_openid_connect_provider" "github" {
  url            = "https://token.actions.githubusercontent.com"
  client_id_list = ["sts.amazonaws.com"]
  thumbprint_list = ["6938fd4d98bab03faadb97b34396831e3780aea1"]
}

# Create IAM role that GitHub Actions can assume
resource "aws_iam_role" "github_actions" {
  name = "github-actions-terraform"
  assume_role_policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Effect = "Allow"
      Principal = { Federated = aws_iam_openid_connect_provider.github.arn }
      Action    = "sts:AssumeRoleWithWebIdentity"
      Condition = {
        StringEquals = {
          "token.actions.githubusercontent.com:aud" = "sts.amazonaws.com"
        }
        StringLike = {
          "token.actions.githubusercontent.com:sub" = "repo:myorg/myrepo:*"
        }
      }
    }]
  })
}

```

Why OIDC over Access Keys:

Feature	Access Keys	OIDC
Credential lifetime	Permanent until rotated	1 hour (auto-expires)
Storage	GitHub Secrets (risk of exposure)	No secrets stored
Rotation	Manual rotation required	No rotation needed
Scope	Anyone with the key can use it	Only specific repo/branch can assume role
Audit	Hard to trace which run used it	Each token has unique session name

Q15: How do you handle secrets in Terraform?

Answer:

Rule: Never put secrets in Terraform code or `.tfvars` files.

Approach 1: AWS Secrets Manager (recommended)

```
# Read secret from Secrets Manager
data "aws_secretsmanager_secret_version" "db_password" {
  secret_id = "production/aurora/master-password"
}

resource "aws_rds_cluster" "aurora" {
  master_password = data.aws_secretsmanager_secret_version.db_password.secret_string
}
```

Approach 2: Environment Variables

```
export TF_VAR_db_password="supersecret"
terraform apply
# Terraform reads TF_VAR_db_password as var.db_password
```

Approach 3: Mark sensitive outputs

```
output "db_endpoint" {
  value      = aws_rds_cluster.aurora.endpoint
  sensitive = true # Won't show in plan/apply output
}
```

State File Protection: - State file WILL contain sensitive values in plain text - **Encrypt state:** S3 backend with KMS encryption - **Restrict access:** IAM policies limit who can read state bucket - **Never commit state to Git:** Add `*.tfstate` to `.gitignore`

4. GitOps & Version Control

Q16: What is GitOps and how do you implement it for infrastructure?

Answer:

GitOps = Git is the single source of truth for infrastructure. All changes go through pull requests.

GitOps Workflow:

```
Developer makes change
|
v
Create feature branch
|
v
Modify Terraform code
|
v
Open Pull Request
|
+-- Automated: terraform fmt check
+-- Automated: terraform validate
+-- Automated: Checkov/tfsec security scan
+-- Automated: terraform plan (output as PR comment)
+-- Manual: Peer review (at least 1 approval)
|
v
Merge to main
|
v
Automated: terraform apply (with environment approval gate)
|
v
State file updated in S3
```

Branch Protection Rules (GitHub):

Rule	Setting
Require PR reviews	Minimum 1 approval
Require status checks	Checkov, tfsec, terraform plan must pass
Require up-to-date branch	Must be rebased on latest main
Dismiss stale reviews	Approval invalidated on new commits
No direct pushes to main	All changes via PR only
Require signed commits	GPG/SSH signature required

Benefits: 1. **Auditability:** Every change has a PR with reviewer, timestamp, and plan output 2. **Rollback:** `git revert` the commit, pipeline applies the revert 3. **Consistency:** No manual changes; infrastructure matches what's in Git 4. **Collaboration:** Everyone can see proposed changes, comment, suggest improvements

Q17: How do you enforce code quality in Terraform PRs?

Answer:

Automated checks in PR pipeline:

```
# .github/workflows/pr-checks.yml
jobs:
  terraform-quality:
    runs-on: ubuntu-latest
    steps:
      # 1. Format check (consistent style)
      - name: Terraform Format
        run: terraform fmt -check -recursive
        # Fails if code is not properly formatted

      # 2. Validate syntax
      - name: Terraform Validate
        run: terraform validate

      # 3. Lint (tflint)
      - name: TFLint
        uses: terraform-linters/setup-tflint@v4
        # Catches: deprecated syntax, unused variables, naming conventions

      # 4. Security scan (Checkov)
      - name: Checkov
        uses: bridgecrewio/checkov-action@v12
        # Catches: public S3 buckets, missing encryption, overly permissive IAM

      # 5. Cost estimation (Infracost)
      - name: Infracost
        uses: infracost/actions/setup@v3
        # Shows: "This PR will increase monthly cost by $45"

      # 6. Documentation (terraform-docs)
      - name: Terraform Docs
        uses: terraform-docs/gh-actions@v1.3.0
        # Auto-generates module documentation in README.md
```

Tool	What It Catches	Example
<code>terraform fmt</code>	Formatting inconsistencies	Indentation, alignment
<code>terraform validate</code>	Syntax errors, type mismatches	Wrong variable type, missing required argument
<code>tflint</code>	AWS-specific issues	Invalid instance type, deprecated resource
<code>Checkov</code>	Security misconfigurations	S3 public access, unencrypted RDS
<code>tfsec</code>	Security best practices	Missing tags, overly permissive IAM
<code>Infracost</code>	Cost impact	"This change adds 3 NAT Gateways (\$97/month)"

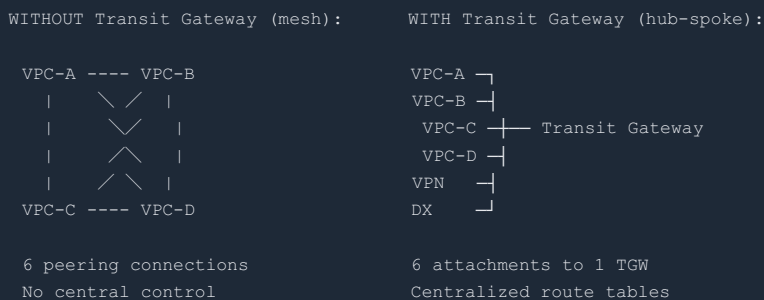
5. AWS Networking Expertise

Q18: Explain Transit Gateway and when you would use it over VPC Peering.

Answer:

VPC Peering: - Direct connection between 2 VPCs - Non-transitive (A<->B and B<->C does NOT mean A<->C) - No bandwidth limits, no extra hop
- Works cross-region and cross-account - **Problem:** With N VPCs, you need $N*(N-1)/2$ peering connections. 10 VPCs = 45 connections!

Transit Gateway: - Hub-and-spoke model: Central hub that all VPCs connect to - **Transitive:** A->TGW->B->TGW->C all reachable - Supports VPCs, VPNs, Direct Connect in one place - Route tables on TGW control which VPCs can communicate



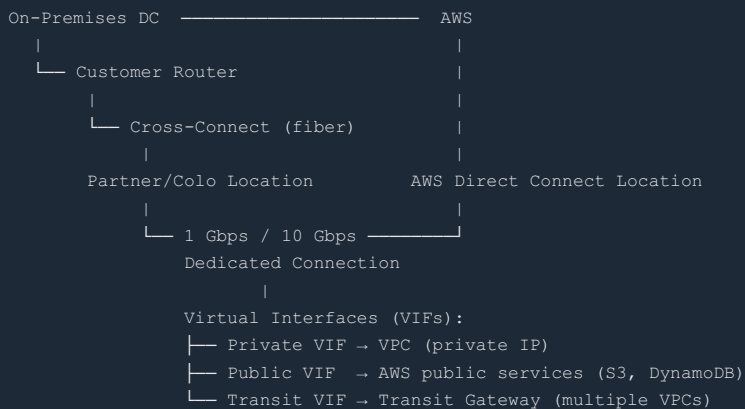
When to Use Each:

Scenario	Choice	Why
2-3 VPCs, simple connectivity	VPC Peering	Simple, no extra cost, no extra hop
5+ VPCs, multi-account	Transit Gateway	Centralized management, scalable
Need VPN + VPC connectivity	Transit Gateway	Single hub for hybrid connectivity
Need traffic inspection between VPCs	Transit Gateway	Route through firewall appliance
Cross-region connectivity	Either	Both support cross-region
Maximum bandwidth	VPC Peering	No bandwidth limit vs TGW 50 Gbps per AZ

Q19: Explain Direct Connect and when you would use it.

Answer:

Direct Connect: A dedicated, private network connection from your on-premises data center to AWS. Bypasses the public internet entirely.



Feature	Direct Connect	VPN (Site-to-Site)
Connection	Dedicated fiber	Over public internet (encrypted)
Bandwidth	1 Gbps, 10 Gbps, 100 Gbps	Up to 1.25 Gbps per tunnel
Latency	Consistent, low	Variable (internet-dependent)
Setup time	Weeks to months	Minutes (fully virtual)
Cost	Port fee + data transfer (lower per GB)	Hourly VPN charges
Encryption	Not encrypted by default (add MACsec)	IPSec encrypted
Resilience	Need 2 connections for HA	2 tunnels per connection (built-in)

When to Use: - Large data transfers (database replication, backups) - Consistent low-latency requirements - Compliance: data must not traverse public internet - Hybrid architectures with heavy on-premises <-> cloud traffic

Best Practice for HA: 2 Direct Connect connections from 2 different locations + VPN as backup.

Q20: Explain Route 53 routing policies. When do you use each?

Answer:

Policy	How It Works	Use Case
Simple	Returns a single record	Single resource (one web server)
Weighted	Distributes traffic by percentage (e.g., 70/30)	Canary deployments, A/B testing
Latency-based	Routes to lowest-latency region	Multi-region applications
Failover	Primary/secondary with health checks	Active-passive DR
Geolocation	Routes based on user's location (country/continent)	GDPR compliance (EU users to EU servers)
Geoproximity	Routes to nearest resource with bias adjustment	Traffic shifting between regions
Multi-value	Returns multiple healthy IPs (up to 8)	Simple load balancing with health checks

Example: Multi-Region Active-Passive Setup:

```
api.myapp.com
|
Route 53 (Failover routing)
|
├─ Primary: us-east-1 ALB (health check: /health)
|   └─ If healthy → Route traffic here
|
└─ Secondary: us-west-2 ALB (failover target)
    └─ If primary fails → Route traffic here
```

Health Checks: - Route 53 can monitor endpoints (HTTP, HTTPS, TCP) - Check interval: 10 or 30 seconds - Failure threshold: 1-10 consecutive failures - Can monitor CloudWatch alarms (for non-HTTP resources like databases) - **Calculated health checks:** Combine multiple health checks (e.g., "healthy if 2 of 3 components are healthy")

Q21: Explain multi-account networking architecture with AWS Organizations.

Answer:


```

AWS Organizations
├── Management Account (billing, org policies)
├── Security OU
│   ├── Log Archive Account (centralized CloudTrail, VPC Flow Logs)
│   └── Security Tooling Account (GuardDuty, Security Hub)
├── Infrastructure OU
│   ├── Network Account (Transit Gateway, Direct Connect, DNS)
│   │   └── Transit Gateway (hub)
│   │       ├── Shared Services VPC (10.0.0.0/16)
│   │       ├── Dev VPC (10.1.0.0/16)
│   │       ├── Staging VPC (10.2.0.0/16)
│   │       └── Production VPC (10.3.0.0/16)
│   └── On-Premises (via Direct Connect)
├── Workload OU
│   ├── Dev Account → Dev VPC attached to TGW
│   ├── Staging Account → Staging VPC attached to TGW
│   └── Production Account → Production VPC attached to TGW
└── Sandbox OU
    └── Sandbox Account (isolated, no TGW access)

```

Key Patterns: - **Centralized networking:** Transit Gateway lives in Network account; shared via RAM (Resource Access Manager) - **Centralized DNS:** Route 53 private hosted zones in Network account; associated with all VPCs - **Centralized egress:** All internet traffic goes through Network account (centralized NAT, firewall) - **Centralized ingress:** ALBs in Network account; backend targets in workload accounts

6. Security & Compliance

Q22: Explain IAM best practices for a production AWS environment.

Answer:

Core Principles:

1. **Least Privilege:** Grant only permissions needed. Start with zero, add as needed.
2. **No Root Account Usage:** MFA on root, use it only for account-level operations.
3. **No Long-Lived Access Keys:** Use IAM roles (OIDC, instance profiles) instead.
4. **MFA Everywhere:** Require MFA for console access and sensitive API calls.

IAM Architecture:

```

Identity Provider (Okta/Azure AD)
|
| (SAML/OIDC Federation)
v
AWS IAM Identity Center (SSO)
|
├── Permission Set: AdminAccess    → Management Account
├── Permission Set: PowerUserAccess → Dev Account
├── Permission Set: ReadOnlyAccess  → Production Account
└── Permission Set: SecurityAudit   → All Accounts

```

Key Policies:

```
// Deny all without MFA
{
  "Effect": "Deny",
  "NotAction": ["iam:CreateVirtualMFADevice", "iam:EnableMFADevice"],
  "Resource": "*",
  "Condition": {
    "BoolIfExists": { "aws:MultiFactorAuthPresent": "false" }
  }
}

// Restrict by source IP
{
  "Effect": "Deny",
  "Action": "*",
  "Resource": "*",
  "Condition": {
    "NotIpAddress": { "aws:SourceIp": ["203.0.113.0/24"] }
  }
}
```

Service Roles (for applications):

Service	IAM Mechanism	Why
EC2	Instance Profile (IAM Role)	No access keys on instance
ECS	Task Role	Per-container permissions
EKS	IRSA (IAM Roles for Service Accounts)	Per-pod permissions
Lambda	Execution Role	Function-level permissions
GitHub Actions	OIDC Federation	No stored credentials

Q23: How do you implement KMS and encryption strategy?

Answer:

KMS Key Types:

Type	Management	Cost	Use Case
AWS Managed Key	AWS manages rotation	Free	S3 default encryption (SSE-S3)
Customer Managed Key (CMK)	You control policies, rotation	\$1/month + API calls	Production databases, sensitive data
Imported Key Material	You provide key material	\$1/month	Regulatory requirement to control key

Encryption Strategy:

Resource	Encryption	Key
S3	SSE-KMS (default bucket encryption)	CMK per project
RDS/Aurora	Encryption at rest (enabled at creation)	CMK
EBS	Default encryption (account-level setting)	CMK
DynamoDB	Encryption at rest (AWS managed or CMK)	CMK for production
ElastiCache	At-rest + in-transit encryption	CMK
Secrets Manager	Automatic encryption	CMK
CloudTrail logs	S3 + KMS encryption	Dedicated CMK

KMS Key Policy Example:

```
{
  "Effect": "Allow",
  "Principal": { "AWS": "arn:aws:iam::123456789012:role/app-role" },
  "Action": [
    "kms:Decrypt",
    "kms:GenerateDataKey"
  ],
  "Resource": "*",
  "Condition": {
    "StringEquals": { "kms:ViaService": "rds.us-east-1.amazonaws.com" }
  }
}
```

Q24: How do you ensure compliance with ISO 27001 and GDPR in AWS?

Answer:

ISO 27001 Controls:

Control Area	AWS Implementation
Access Control	IAM least privilege, MFA, SSO, no root usage
Asset Management	AWS Config (inventory), resource tagging
Cryptography	KMS encryption at rest, TLS in transit
Operations Security	CloudTrail (audit), GuardDuty (threat detection)
Network Security	VPC isolation, security groups, WAF
Incident Management	CloudWatch alarms, SNS, PagerDuty integration
Backup	Automated snapshots, cross-region replication
Compliance	AWS Audit Manager, Config Rules, Security Hub

GDPR Compliance:

GDPR Requirement	AWS Implementation
Data residency	Choose EU regions (eu-west-1, eu-central-1)
Right to erasure	Implement data deletion APIs; DynamoDB TTL
Data encryption	KMS encryption everywhere
Access logging	CloudTrail + VPC Flow Logs
Data minimization	Collect only what's needed; archive/delete old data
Breach notification	GuardDuty + automated alerting (72-hour window)
Data Processing Records	CloudTrail audit trails in S3 (immutable)

AWS Tools for Compliance:

Tool	Purpose
AWS Config	Track resource configurations, detect non-compliant resources
Config Rules	Automated compliance checks (e.g., "all S3 buckets must be encrypted")
Security Hub	Aggregated security findings, compliance scores
Audit Manager	Automated evidence collection for audits
CloudTrail	API call logging (who did what, when)
GuardDuty	ML-based threat detection

7. Cost Optimization

Q25: Design a cost optimization strategy for AWS infrastructure.

Answer:

Savings Mechanisms:

Strategy	Savings	Best For	Commitment
On-Demand	0% (baseline)	Unpredictable workloads	None
Reserved Instances (1yr)	~30-40%	Steady-state databases (RDS, ElastiCache)	1 or 3 year
Savings Plans (1yr)	~20-30%	Compute (EC2, Fargate, Lambda)	1 or 3 year
Spot Instances	~60-90%	Batch processing, CI/CD runners, dev/test	Can be interrupted
Graviton (ARM)	~20%	EC2, RDS, ElastiCache (arm64 support)	None

Quick Wins:

- Right-size instances
 - AWS Compute Optimizer → shows which instances are oversized
 - Example: m5.2xlarge (CPU 15%) → m5.large saves \$200/month
- Delete unused resources
 - Unattached EBS volumes (\$0.10/GB/month)
 - Idle Elastic IPs (\$3.65/month each)
 - Old snapshots (accumulate over time)
 - Unused NAT Gateways (\$32/month + data transfer)
- S3 lifecycle policies
 - Move to IA after 30 days (50% cheaper)
 - Move to Glacier after 90 days (80% cheaper)
 - Delete after 365 days
- Dev/staging shutdown
 - Lambda + EventBridge: Stop instances at 7 PM, start at 8 AM
 - Save 65% on compute (only running 9 hours/day on weekdays)
- VPC endpoints instead of NAT
 - S3 Gateway endpoint: FREE (vs NAT Gateway: \$0.045/GB)
 - Interface endpoints: \$0.01/hr + \$0.01/GB (often cheaper than NAT)

Cost Monitoring:

```
AWS Budgets:
├─ Monthly budget: $50,000
│  └─ Alert at 80% ($40,000)
│  └─ Alert at 100% ($50,000)
│     └─ Auto-action: Deny new resource creation at 120%
├─ Per-team budget (via tags: Team=platform)
│  └─ Alert team lead when their spend exceeds threshold
├─ Reserved Instance utilization alert
│  └─ Alert if RI utilization < 80% (wasting reservation)
```

8. Multi-Environment Management

Q26: How do you manage Dev, QA, Staging, and Production environments consistently?

Answer:

Architecture:

```

Git Repository
├── modules/           # Shared, reusable modules (same code for all envs)
│   ├── networking/
│   ├── compute/
│   └── database/
└── environments/
    ├── dev/
    │   ├── main.tf      # Same module calls
    │   └── terraform.tfvars # DIFFERENT values
    │       instance_type = "t3.small"
    │       db_instance_class = "db.t3.medium"
    │       min_capacity = 1
    │       multi_az = false
    ├── staging/
    │   └── terraform.tfvars
    │       instance_type = "t3.medium"
    │       db_instance_class = "db.r6g.large"
    │       min_capacity = 2
    │       multi_az = true
    └── production/
        └── terraform.tfvars
            instance_type = "c6i.xlarge"
            db_instance_class = "db.r6g.2xlarge"
            min_capacity = 3
            multi_az = true

```

Environment Differences:

Aspect	Dev	Staging	Production
Instance size	t3.small	t3.medium	c6i.xlarge
Multi-AZ	No	Yes	Yes
Auto Scaling	1-2	2-4	3-15
Database	db.t3.medium, single AZ	db.r6g.large, multi-AZ	db.r6g.2xlarge, multi-AZ + read replicas
NAT Gateway	1 (cost saving)	2 (HA)	2 (HA)
Backup retention	7 days	14 days	35 days
Monitoring	Basic	Detailed	Detailed + custom dashboards
Access	Dev team, wide	Dev + QA, moderate	Ops only, restricted

Promotion Strategy:

```

Code change → Dev (auto-deploy)
               → Manual approval
               → Staging (auto-deploy, run integration tests)
               → Manual approval (change advisory board)
               → Production (deploy with canary/rolling)

```

9. Cross-Functional Collaboration

Q27: How do you collaborate with DevOps, Security, and Development teams?

Answer:

My Approach:

1. **Shared Responsibility Model:**
2. Security team defines policies (e.g., "all S3 buckets must be encrypted")
3. I implement policies as Terraform modules and Checkov rules

4. Dev teams use the modules - compliance is automatic, not manual

5. **Architecture Review Board (ARB):**

6. Weekly 1-hour session with representatives from each team

7. Review new designs, proposed changes, security findings

8. Decisions documented as ADRs (Architecture Decision Records) in Git

9. **Inner-Source Terraform Modules:**

10. I build reusable, opinionated modules that embed best practices

11. Example: `module "secure-s3"` automatically enables encryption, versioning, access logging, blocks public access

12. Teams use modules via Git tags: `source = "git:...?ref=v2.0.0"`

13. **Blameless Culture:**

14. Post-incident reviews focus on system improvements, not individual blame

15. Share findings openly in Confluence/wiki

16. **Office Hours:**

17. Weekly open session where any team can bring architecture questions

18. No formal agenda needed — drop-in format

10. Mentorship & Innovation

Q28: How do you drive technical upskilling and innovation in your team?

Answer:

1. **Architecture Katas:** Monthly workshops where engineers design systems for realistic scenarios. Present and receive peer feedback. Builds architecture thinking across the team.
 2. **Certification Support:** Sponsor and coach team members for AWS certifications. Create study groups that meet weekly. Share exam tips from my own certifications (SAA, CCP, GCP-PCA).
 3. **Innovation Sprints (20% Time):** Dedicate 1 sprint per quarter for engineers to explore new technologies. Past examples:
 4. Evaluating Karpenter vs Cluster Autoscaler
 5. Building a cost dashboard with CUR + Athena + QuickSight
 6. Testing Terraform 1.5 import blocks for migration
 7. **Internal Tech Talks:** Bi-weekly 30-minute talks. Everyone presents at least once per quarter. Topics range from deep-dives on specific AWS services to lessons learned from incidents.
 8. **Tech Radar:** Quarterly review of technologies with the team. Categorize as Adopt/Trial/Assess/Hold. Creates shared vocabulary for technology decisions.
-

11. Scenario-Based Questions

Q29: You run `terraform apply` and it tries to destroy a production database. What do you do?

Answer:

Immediate Action: Cancel the apply! (Ctrl+C before the destroy starts)

Then investigate:

```
# Step 1: Why did Terraform want to destroy it?
terraform plan # Read the plan carefully

# Common causes:
# a) Someone changed the resource name in code
#   → resource "aws_rds_cluster" "aurora" was renamed to "main"
#   → Terraform sees: destroy "aurora", create "main"
#   → Fix: terraform state mv aws_rds_cluster.aurora aws_rds_cluster.main

# b) State file corruption
#   → Fix: terraform state pull > backup.tfstate
#   → Then: terraform import

# c) Variable changed that forces replacement
#   → e.g., db_name changed (forces new DB)
#   → Fix: revert the variable change, use lifecycle block
```

Prevention (lifecycle blocks):

```
resource "aws_rds_cluster" "aurora" {
  # ... config ...

  lifecycle {
    prevent_destroy = true # Terraform will ERROR instead of destroying
  }
}
```

Prevention (state move for renames):

```
# When renaming a resource:
terraform state mv aws_rds_cluster.old_name aws_rds_cluster.new_name
# Then update code. Plan should show: 0 to add, 0 to change, 0 to destroy
```

Q30: A developer reports they can't connect from EC2 in a private subnet to RDS. How do you troubleshoot?

Answer:

Systematic Troubleshooting (Layer by Layer):

```

Step 1: Security Group Check
EC2 SG → Does it allow OUTBOUND to port 5432?
RDS SG → Does it allow INBOUND on port 5432 FROM the EC2 SG?

Fix: Add inbound rule to RDS SG:
Type: PostgreSQL
Port: 5432
Source: sg-ec2-security-group

Step 2: NACL Check
EC2 subnet NACL → Outbound rule allows port 5432?
RDS subnet NACL → Inbound rule allows port 5432?
RDS subnet NACL → Outbound rule allows ephemeral ports (1024-65535)?
EC2 subnet NACL → Inbound rule allows ephemeral ports?

Step 3: Route Table Check
EC2 subnet RT → Has route to RDS subnet CIDR? (should be "local" route)
If different VPC → Need VPC peering/TGW route

Step 4: RDS Configuration
Is RDS in the same VPC?
Is RDS subnet group using the correct subnets?
Is RDS publicly accessible = false? (good, but means private access only)

Step 5: DNS Resolution
Can EC2 resolve the RDS endpoint?
→ nslookup mydb.cluster-xxx.us-east-1.rds.amazonaws.com
VPC DNS resolution enabled? (enableDnsSupport = true)

Step 6: IAM Authentication (if using IAM auth)
Does the EC2 instance role have rds-db:connect permission?

```

Q31: How do you handle a Terraform state file that got corrupted?

Answer:

```

# Step 1: Check S3 versioning (if enabled)
aws s3api list-object-versions \
  --bucket company-terraform-state \
  --prefix production/terraform.tfstate

# Step 2: Download the last known good version
aws s3api get-object \
  --bucket company-terraform-state \
  --key production/terraform.tfstate \
  --version-id "xxxxx" \
  state-backup.tfstate

# Step 3: Restore the good state
aws s3 cp state-backup.tfstate s3://company-terraform-state/production/terraform.tfstate

# Step 4: If no good version exists, rebuild state from scratch
terraform import aws_vpc.main vpc-0abc123
terraform import aws_subnet.public_a subnet-0def456
# ... import every resource one by one

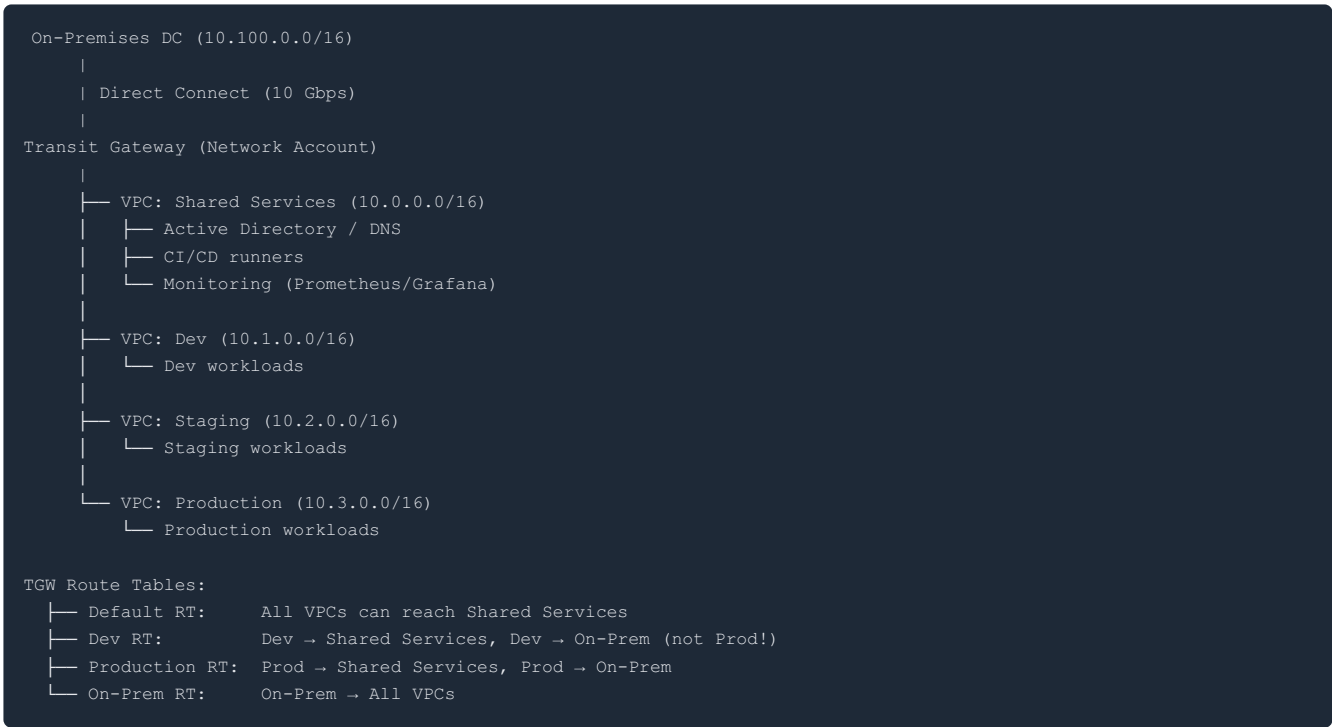
# Step 5: Verify
terraform plan
# Should show: No changes. Infrastructure is up-to-date.

```

Prevention: - S3 bucket versioning: ALWAYS enabled - DynamoDB locking: ALWAYS enabled - Regular state backups (additional to S3 versioning) - Never manually edit state files

Q32: Design networking for a company that needs to connect 4 AWS accounts with an on-premises data center.

Answer:



CIDR Planning:

Account	VPC CIDR	Purpose
Network	10.0.0.0/16	Shared Services, Transit Gateway
Dev	10.1.0.0/16	Development workloads
Staging	10.2.0.0/16	Staging workloads
Production	10.3.0.0/16	Production workloads
On-Premises	10.100.0.0/16	Data center

Key: CIDRs must NOT overlap. Plan CIDR allocation ahead of time.

12. Quick Reference Cheat Sheet

Key AWS Limits

Resource	Default Limit	Adjustable?
VPCs per region	5	Yes
Subnets per VPC	200	Yes
Security Groups per VPC	2,500	Yes
Rules per Security Group	60 inbound + 60 outbound	Yes
Elastic IPs per region	5	Yes
IGW per VPC	1	No
NAT Gateway per AZ	5	Yes
Route tables per VPC	200	Yes
Routes per route table	50	Yes (max 1,000)
VPC Peering per VPC	50	Yes (max 125)
Transit Gateway attachments	5,000	No

CIDR Quick Math

CIDR	IPs Total	Usable (AWS)	Common Use
/16	65,536	65,531	VPC
/20	4,096	4,091	Large subnet
/24	256	251	Standard subnet
/28	16	11	Minimum subnet (for small resources)

Terraform Commands Cheat Sheet

```
terraform init           # Initialize, download providers
terraform plan           # Preview changes
terraform apply          # Apply changes
terraform destroy        # Destroy all resources
terraform fmt -recursive # Format code
terraform validate       # Validate syntax
terraform state list     # List resources in state
terraform state show <resource> # Show resource details
terraform state mv <old> <new> # Rename resource in state
terraform import <resource> <id> # Import existing resource
terraform output         # Show outputs
terraform workspace list # List workspaces
terraform force-unlock <id> # Release stuck lock
terraform plan -target=<resource> # Plan for specific resource only
terraform apply -refresh-only # Refresh state without changes
```

Common GitHub Actions Syntax

```
on:
  push:
    branches: [main]
    paths: ['terraform/**'] # Only trigger on terraform changes
  pull_request:
    branches: [main]

jobs:
  plan:
    runs-on: ubuntu-latest
    permissions:
      id-token: write # For OIDC
    steps:
      - uses: actions/checkout@v4
      - uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: ${ secrets.AWS_ROLE_ARN }
          aws-region: us-east-1
      - uses: hashicorp/setup-terraform@v3
      - run: terraform init && terraform plan
```

Interview Day Tips

1. **Lead with experience:** "In my 22+ years, I've designed VPC architectures for enterprise clients including Advantest, HPE, and Nokia..."
2. **Draw diagrams:** Ask for whiteboard. Visual thinking is expected from architects.
3. **Mention Terraform specifics:** State locking, modules, `for_each`, `lifecycle`, `import` — shows hands-on depth.
4. **Security-first answers:** Always mention encryption, least privilege, and compliance in every architecture answer.
5. **Cost consciousness:** Mention cost implications of design decisions (NAT Gateway charges, cross-AZ data transfer).
6. **This is a 7-12 year role but you have 22+:** Position as "I bring senior leadership perspective while still being hands-on with Terraform and GitHub Actions daily."